

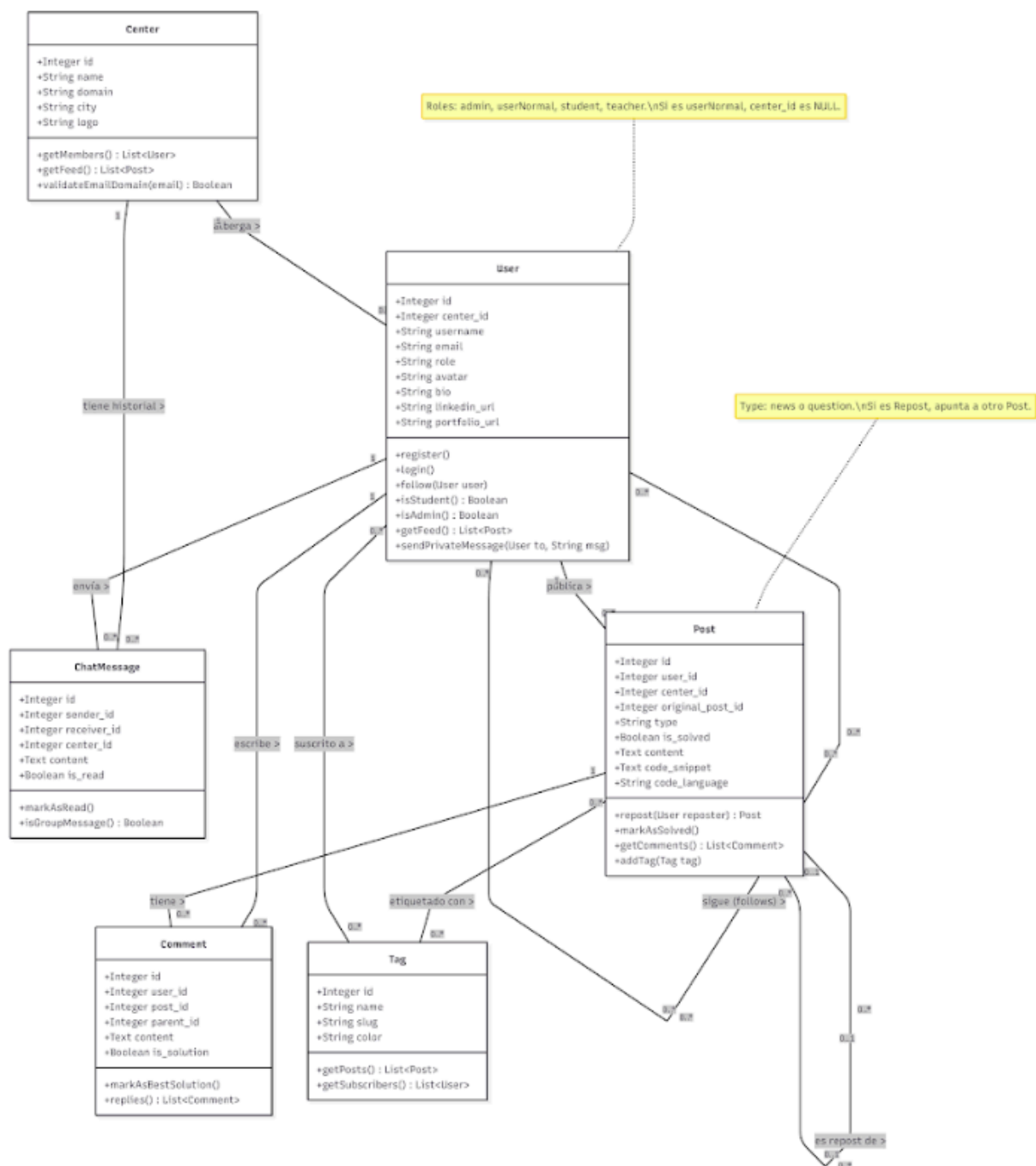
Documentació Tècnica Codex



Arquitectura de l'aplicació:	3
Rutes de l'aplicació:	5
Arquitectura General	5
API Principal (Laravel)	5
Servei de WebSockets (Node.js)	7
Servei de Moderació d'IA (Node.js)	8
Esquema d'esdeveniments de comunicació:	8
Base de dades:	9
Esquema de components de frontend:	10
Estructura (resum)	10
Descripció dels components	12
Descripció dels components	13
Documentació de codi frontend:	13
Plantilla Principal (entrada de l'app)	13
Seccions Clau	14
Gestió de Dades	15
const [user, setUser] = useState(null);	15
const [emailVerified, setEmailVerified] = useState(false);	15
const [authMessage, setAuthMessage] = useState(null);	15
Mètodes Principals	15
Característiques Visuals	16
Funcionalitats Destacades	16
Documentació de codi Laravel:	18
Plantilla Principal (serveis base)	18
Seccions Clau	19
Gestió de Dades	20
Mètodes Principals	20
Característiques Visuals	20
Funcionalitats Destacades	20
Documentació de codi Socket.io:	21
Plantilla Principal (servidor sockets)	21
Seccions Clau	21
Gestió de Dades	22
Mètodes Principals	22
Característiques Visuals	22
Funcionalitats Destacades	22
AI-moderation (microservei):	23
Plantilla Principal	23
Seccions Clau	23
Gestió de Dades	24
Mètodes Principals	25
async function getClassifier() {	25
classifierPromise = pipeline('text-classification', MODEL_ID, { quantized: true });	25
return classifierPromise;}	25

Característiques Visuals	25
Funcionalitats Destacades	25
Dockers:	26
Visió general del stack	26
Desplegament a DESENVOLUPAMENT	26
Desplegament a DESENVOLUPAMENT	27
Persistència i dades	28
Ordres útils	28

vements importants i



Rutes de l'aplicació:

Arquitectura General

El backend està compost per tres serveis principals:

- **API Principal (Laravel):** Gestiona la lògica de negoci, la persistència de dades (base de dades relacional) i l'autenticació.
- **Servei de WebSockets (Node.js + [Socket.io](#)):** Gestiona la comunicació en temps real per al xat, les notificacions i l'inici/fi de trucades (WebRTC).
- **Servei de Moderació d'IA (Node.js):** Microservei aïllat encarregat d'avaluar la toxicitat o llenguatge ofensiu dels textos creats pels usuaris.

API Principal (Laravel)

Ruta del projecte: /api

Fitxers de rutes principals: api/routes/api.php i api/routes/web.php

Totes les peticions externes cap a l'API Laravel tenen el prefix /api. A continuació s'agrupen les rutes principals per funcionalitat:

Salut de l'API

- GET /health: Endpoint per comprovar l'estat del servei.
- GET /sitemap: Generació del sitemap del lloc web.

Autenticació i Usuaris

- POST /register: Registre d'un nou usuari.
- POST /login: Inici de sessió (retorna token Sanctum).
- POST /logout: Tancament de sessió (requereix token).
- GET /me: Retorna la informació de l'usuari actual autenticat.
- POST /check-domain: Comprova si el domini del correu està vinculat a algun centre.
- GET /auth/google/redirect/POST /auth/google/callback: Gestió del flux d'OAuth amb Google.
- GET /email/verify/{id}/{hash}: Verifica l'adreça de correu de l'usuari.
- POST /password/forgo/POST /password/reset: Flux de recuperació de contrasenya.
- PUT /password/update: Actualització de contrasenya (requereix autenticació).

Gestió de Publicacions i Interaccions (CRUD)

- GET /posts: Llista de publicacions públiques.
- GET /posts/{post}: Detall d'una publicació.
- POST /posts: Creació d'una nova publicació.
- PUT /posts/{post}/DELETE /posts/{post}: Edició o eliminació d'una publicació.
- POST /posts/{post}/repost: Compartir (repost) d'una publicació.
- GET /feed/following: Feed de les persones i etiquetes que l'usuari segueix.
- POST /comments, PUT /comments/{comment}, DELETE /comments/{comment}: CRUD de comentaris.
- PATCH /comments/{comment}/solution: Marca un comentari com la solució d'un dubte.

- POST /interactions: Alterna l'estat d'un "M'agrada" o d'una publicació "Guardada".
- GET /bookmarks/ GET /liked: Llistat de publicacions guardades o a les que s'ha donat m'agrada.

Seguidors i Xarxa

- GET /users/{user}/followers / GET /users/{user}/following: Llistat de seguidors / seguits.
- POST /users/{user}/follow: Seguir o deixar de seguir a un usuari.
- GET /follow-requests, POST /follow-requests/{follower}/accept, POST /follow-requests/{follower}/reject: Gestió de sol·licituds de seguiment en comptes privats.

Perfils i Etiquetes

- GET /profile/{username}: Veure un perfil complet.
- PUT /profile: Actualitzar la informació del perfil de l'usuari autenticat.
- GET /tags: Llista d'etiquetes disponibles o populars.
- POST /tags/{tag}/follow, PATCH /tags/{tag}/notify: Seguir etiquetes i activar notificacions.

Missatgeria (Xat)

- GET /chat/conversations: Llistat de converses de l'usuari.
- GET /chat/conversations/{userId}: Historial de missatges amb un altre usuari.
- POST /chat/messages: Emmagatzematge d'un missatge de xat enviat.
- POST /groups / GET /groups: Creació i obtenció de grups de xat.

"Walled Garden" i Gestió de Centres

Aquest apartat gestiona l'espai privat d'un centre educatiu.

- GET /centers: Llista de centres donats d'alta.
- GET /center/posts, GET /center/tags: Obté el contingut exclusiu generat dintre de la comunitat d'un centre.
- POST /center-requests: Sol·licita entrar a formar part d'un centre.
- PUT /centers/{center}: (Només Professorat) Actualitza informació del centre.
- GET /center/members: (Només Professorat) Llista de membres del centre.
- PATCH /center/members/{user}/role, PATCH /center/members/{user}/block: (Només Professorat) Gestió de permisos, moderació i bloqueig d'usuaris dintre del centre.

Funcions d'Administrador Global

Totes aquestes rutes només són accessibles pel rol *Admin*.

- GET /admin/stats: Dades generals del Dashboard d'administració.
- GET /admin/users, PUT /admin/users/{user}, DELETE /admin/users/{user}: Gestió global d'usuaris.
- POST /admin/users/{user}/ban / POST /admin/users/{user}/unban: Suspènre l'accés d'un usuari.
- PATCH /center-requests/{centerRequest}/approve / reject: Aprovar o denegar la creació d'un nou centre educatiu en base al justificant presentat.

Servei de WebSockets (Node.js)

Ruta del projecte: /socket

Fitxer d'entrada principal: socket/index.js

Aquest servei proveeix de funcionalitat en temps real connectant l'aplicació client (React) a través de la llibreria `socket.io`.

Esdeveniments (Events) del Socket

El servidor gestiona els següents esdeveniments emesos per o dirigits als usuaris connectats:

- **Gestió de Sales (Rooms):**
 - ``join``, ``join-admin``: Subscripció al canal personal o al panell d'administrador.
 - ``join-post``, ``leave-post``: Subscripció als esdeveniments d'una publicació concreta (ex: algú acaba de comentar-hi).
 - ``join-chat``, ``leave-chat``, ``join-group``: Subscripció als canals de missatgeria directa o grupal.
- **Xat en Temps Real::**
 - ``typing``: Avís que un usuari està escrivint.
 - ``send-message``: Notifica a l'altre extrem l'arribada d'un missatge i l'emmagatzema.
 - ``mark-read``: Actualitza l'estat d'un missatge a "llegit" per als participants de la conversa.
- **Senyalització WebRTC (Trucades de veu i vídeo):**

Aquest servei no actua com a servidor multimèdia de vídeo, sinó com a servidor de **signaling** per negociar la connexió peer-to-peer (WebRTC) entre dos usuaris.

 - ``call-user``: Usuari A vol iniciar una trucada amb l'Usuari B.
 - ``make-answer``: Usuari B accepta i contesta a la trucada.
 - ``ice-candidate``: Intercanvi d'informació de xarxa.
 - ``end-call`` / ``reject-call``: Finalització o rebuig d'una trucada.
 - ``video-toggle``, ``audio-toggle``: Sincronització de l'estat (mut/sense càmera).

Servei de Moderació d'IA (Node.js)

Ruta del projecte: /ai-moderation

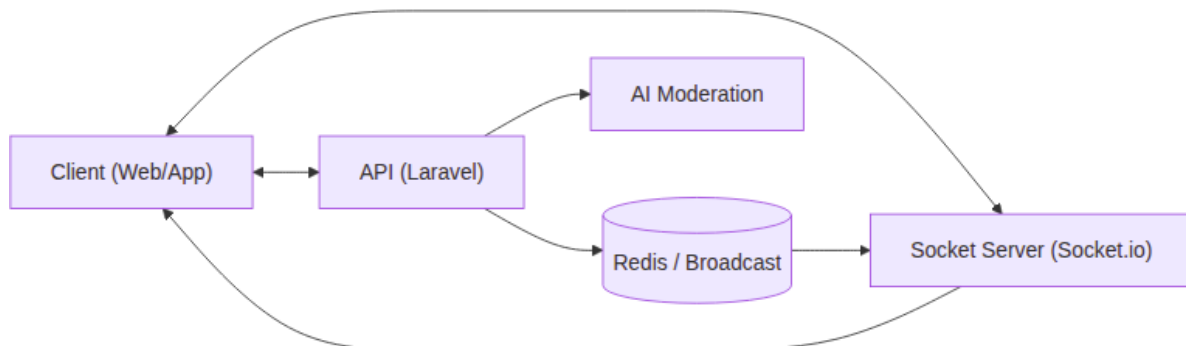
Fitxer d'entrada principal: ai-moderation/index.js

És un microservei API (servidor Express independent) encarregat exclusivament d'avaluar si un contingut de text és apropiat o no.

Endpoints del Microservei

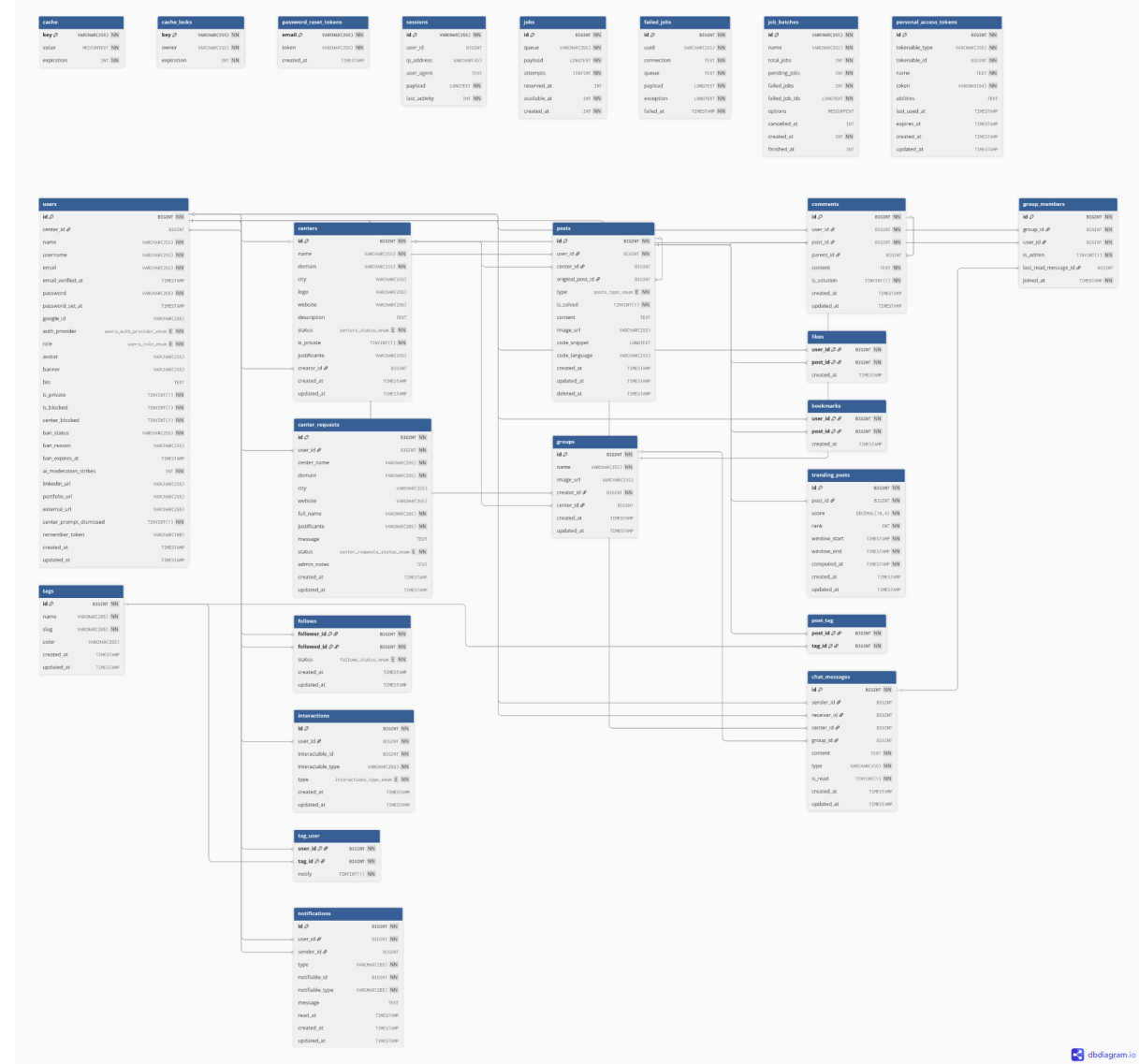
- **GET /health:** Comprova l'estat d'execució d'aquest microservei.
- **POST /moderate:**
 - **Funció:** Rep una càrrega útil (text a moderar).
 - **Procés:** Analitza el contingut mitjançant un model d'Intel·ligència Artificial i detecta discurs d'odi, assetjament o contingut nsfw.
 - **Sortida:** Retorna una classificació detallant el nivell de "toxicitat" per establir si s'aprova o si es bloqueja la publicació o el comentari.
 - **Seguretat:** Aquest servei només accepta peticions de la pròpia API principal (Laravel) validant-ho a través d'una clau d'autenticació interna.

Esquema d'esdeveniments de comunicació:



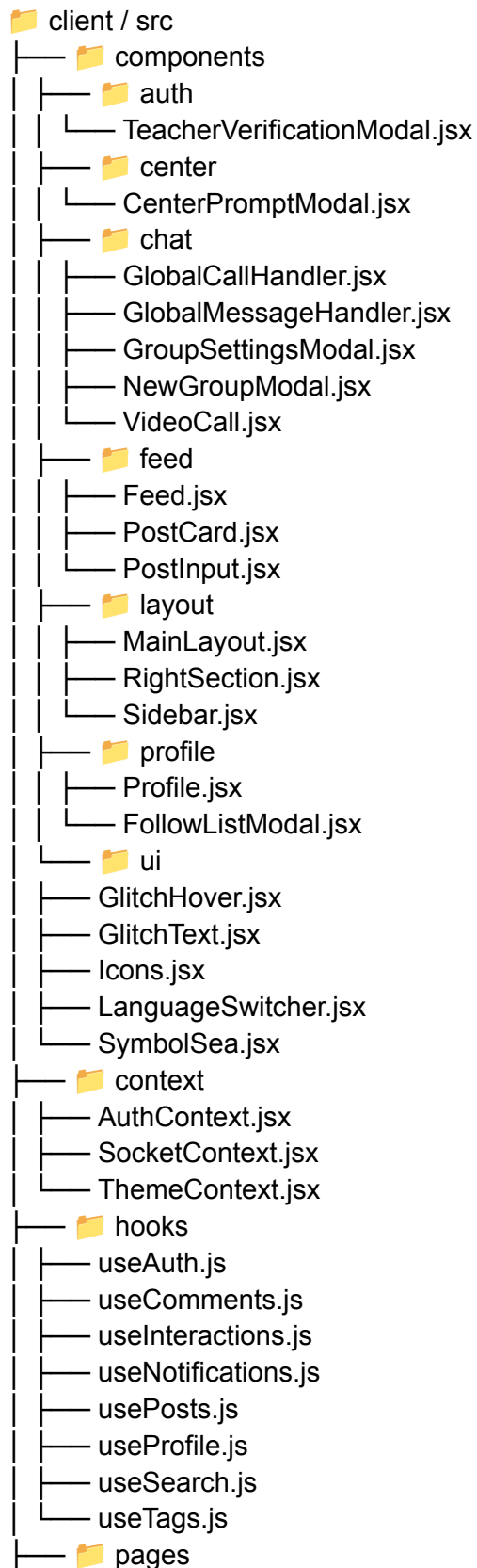
Base de dades:

doc/schema.sql



Esquema de components de frontend:

Estructura (resum)



- Home.jsx
- Explore.jsx
- PostDetail.jsx
- ProfilePage.jsx
- Notifications.jsx
- Messages.jsx
- CenterHub.jsx
- Settings.jsx
- Landing.jsx
- GoogleCallback.jsx
- EmailVerification.jsx
- ForgotPassword.jsx
- ResetPassword.jsx
- LegalDocs.jsx
-  admin
 - AdminLayout.jsx
 - AdminOverview.jsx
 - AdminUsers.jsx
 - AdminUserPosts.jsx
 - AdminModeration.jsx
 - AdminCenters.jsx
 - AdminRequests.jsx
-  router
 - index.jsx
 - ProtectedRoute.jsx
-  services
- api.js
- auth/center/posts/comments/etc.

Descripció dels components

Components principals (layout i navegació)

- **MainLayout.jsx**: Contenidor global de l'app (sidebar, contingut i widgets), gestiona toasts i modals globals.
- **Sidebar.jsx**: Navegació principal i accessos ràpids.
- **RightSection.jsx**: Widgets laterals (tendències, suggeriments, etc.).

Components del feed

- **Feed.jsx**: Llista principal de posts amb tabs i scroll infinit.
- **PostInput.jsx**: Editor de publicacions (text, codi, imatge, enllaç).
- **PostCard.jsx**: Targeta d'un post amb interaccions (likes, comments, share).

Components de perfil

- **Profile.jsx**: Vista de perfil amb dades, posts i pestanyes.
- **FollowListModal.jsx**: Modal per veure seguidors/seguint.

Components de chat

- **GlobalCallHandler.jsx**: Gestiona entrades de trucades (WebRTC).
- **VideoCall.jsx**: UI de video/trucada.
- **GlobalMessageHandler.jsx**: Gestió global d'events de missatges.
- **NewGroupModal.jsx**: Crear grups de chat.
- **GroupSettingsModal.jsx**: Configuració i gestió del grup.

Components d'autenticació i centre

- **TeacherVerificationModal.jsx**: Validació de professor i upload de justificants.
- **CenterPromptModal.jsx**: Modal per crear o vincular centre.

Components UI

- **LanguageSwitcher.jsx**: Selector d'idioma.
- **GlitchText.jsx** / **GlitchHover.jsx**: Efectes visuals de text.
- **Icons.jsx**: Pack d'icones reutilitzables.
- **SymbolSea.jsx**: Fons decoratiu.

Pàgines (rutes principals)

- **Home.jsx**: Feed principal.
- **Explore.jsx**: Cerca i descobriment.
- **PostDetail.jsx**: Detall d'un post.
- **ProfilePage.jsx**: Perfil públic.
- **Notifications.jsx**: Notificacions.
- **Messages.jsx**: Xat i converses.
- **CenterHub.jsx**: Hub del centre educatiu.
- **Settings.jsx**: Configuració d'usuari.
- **Landing.jsx**: Login/registre.
- **GoogleCallback.jsx**: Callback OAuth.
- **EmailVerification.jsx**: Verificació d'email.
- **ForgotPassword.jsx** / **ResetPassword.jsx**: Recuperació de contrasenya.
- **LegalDocs.jsx**: Termes i privacitat.

Pàgines d'admin

- **AdminLayout.jsx**: Layout d'administració.
- **AdminOverview.jsx**: Resum i KPIs.
- **AdminUsers.jsx**: Gestió d'usuaris.
- **AdminUserPosts.jsx**: Posts d'un usuari.
- **AdminModeration.jsx**: Moderació de contingut.
- **AdminCenters.jsx**: Gestió de centres.
- **AdminRequests.jsx**: Sol·licituds de centre.

Descripció dels components

- L'arrencada carrega els contextos globals i el router.
- El router decideix la pàgina i la insereix dins del layout principal.
- Les pàgines consumeixen hooks i serveis; els components visuals renderitzen dades.
- El SocketContext propaga events en temps real a tot el frontend.

Documentació de codi frontend:

Plantilla Principal (entrada de l'app)

Codi base (punt d'entrada)

```
<React.StrictMode>
  <HelmetProvider>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </HelmetProvider>
</React.StrictMode>
```

La plantilla principal inicialitza React, el router i el provider de metadades.

Seccions Clau

Providers globals (autenticació, sockets, tema)

```
<ThemeProvider>
  <AuthProvider>
    <SocketProvider>
      <AppRouter />
    </SocketProvider>
  </AuthProvider>
</ThemeProvider>
```

Aquesta secció agrupa la configuració global abans de renderitzar rutes.

Router centralitzat (rutes i proteccions)

```
<Route element={<ProtectedRoute />}>
  <Route element={<VerifiedRoute />}>
    <Route path="/" element={<Home />} />
    <Route path="/messages" element={<Messages />} />
  </Route>
</Route>
```

Defineix rutes protegides i verifica email abans d'accedir.

Layout principal (estructura visual)

```
<Sidebar />
<main className="main-content">
  <div className="main-content__container">
    <Outlet />
  </div>
</main>
<RightSection />
```

El layout distribueix navegació lateral, contingut central i widgets.

Gestió de Dades

Estat global d'autenticació

```
const [user, setUser] = useState(null);

const [emailVerified, setEmailVerified] = useState(false);

const [authMessage, setAuthMessage] = useState(null);
```

L'estat d'usuari i verificació es manté a nivell global.

Socket i comptadors de no llegits

```
const [unreadCount, setUnreadCount] = useState(0);
const [unreadMessagesCount, setUnreadMessagesCount] = useState(0);
```

Els comptadors s'actualitzen via events temps real.

Mètodes Principals

Login / Logout

```
const login = async (email, password) => {
  const response = await api.post("/login", { email, password });
  localStorage.setItem("token", response.data.token);
  setUser(response.data.user);
};
```

Autenticació amb token i persistència a localStorage.

Crear post amb feedback

```
const handleCreatePost = async (postData) => {
  const result = await createPost(postData);
  if (result.success) setIsPostModalOpen(false);
};
```

Crea post i tanca modal si l'operació és correcta.

Scroll infinit

```
const observer = new IntersectionObserver(entries => {  
  if (entries[0].isIntersecting && hasMore && !loading) loadMore();  
});
```

Carrega més posts quan el “sentinel” entra en pantalla.

Característiques Visuals

Skeleton Loader

```
const PostSkeleton = () => (  
  <div className="post-skeleton">  
    <div className="post-skeleton__avatar" />  
    <div className="post-skeleton__content" />  
  </div>  
) ;
```

Mostra placeholders mentre es carreguen dades.

Tabs actives amb classes dinàmiques

```
<button className={`feed__tab ${activeTab === "all" ?  
"feed__tab--active" : ""}`}>  
  Para ti  
</button>
```

Canvia l'estat visual segons la pestanya activa.

Funcionalitats Destacades

Temps real (notificacions i missatges)

```
socketService.onMessage(handleNewMessage);  
socketService.onNotification(handleNotification);
```

Reacciona a events del servidor i actualitza UI.

Multiidioma

```
const { t } = useTranslation();  
<h2>{t("feed.welcome_title")}</h2>
```

Textos dinàmics segons l'idioma.

Highlight de codi als posts

```
hljs.highlightElement(codeBlock);
```

Destaca blocs de codi dins del contingut.

Documentació de codi Laravel:

Plantilla Principal (serveis base)

Servei de moderació (exemple real)

```
<?php
public function moderatePost(?string $content, ?string $codeSnippet,
bool $hasImage = false): array
{
    $payload = [
        'context' => 'post',
        'content' => $content,
        'code_snippet' => $codeSnippet,
        'image_present' => $hasImage,
    ];

    $response = $request->post("{ $baseUrl }/moderate", $payload);

    return [
        'allowed' => (bool) ($data['allowed'] ?? true),
        'score' => (float) ($data['score'] ?? 0.0),
        'severity' => (string) ($data['severity'] ?? 'low'),
        'reason' => (string) ($data['reason'] ?? 'No reason provided.'),
        'categories' => is_array($data['categories'] ?? null) ?
$data['categories'] : [],
    ];
}
```

Servei que encapsula una crida HTTP a un microservei d'IA.

Fitxer: AiModerationService.php

Seccions Clau

1. Migracions (esquema de base de dades)

```
<?php
Schema::create('posts', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');

    $table->foreignId('center_id')->nullable()->constrained()->onDelete('cascade');

    $table->enum('type', ['news', 'question'])->default('news');
    $table->text('content')->nullable();
    $table->timestamps();
    $table->softDeletes();
});
```

Defineix l'estructura de taules i relacions.

Fitxer: 2025_02_17_000000_create_full_schema.php

2. Configuració BD (connexions)

```
<?php
'connections' => [
    'mysql' => [
        'driver' => 'mysql',
        'host' => env('DB_HOST', '127.0.0.1'),
        'database' => env('DB_DATABASE', 'laravel'),
        'username' => env('DB_USERNAME', 'root'),
        'password' => env('DB_PASSWORD', ''),
    ],
],
```

Centralitza els paràmetres de connexió.

Fitxer: database.php

Gestio de Dades

- **Fonts:** API REST (controllers) + serveis interns.
- **Persistencia:** MySQL via Eloquent + migrations.
- **Integracions:** microservei d'IA per moderacio.

Metodes Principals

- **Moderacio de contingut:** moderatePost() al servei.
- **Persistencia:** taules i relacions definides per migracions.
- **Config BD:** entorns i connexions a database.php.

Caracteristiques Visuals

No aplica (backend). Equivalent: **logs, errors i temps de resposta.**

Funcionalitats Destacades

- Moderacio IA integrada amb fallback i “fail-open”.
- Esquema complet amb relacions i indexes.
- Configuracio MySQL centralitzada.

Documentació de codi Socket.io:

Plantilla Principal (servidor sockets)

```
const io = new Server(server, {
  cors: { origin: process.env.CORS_ORIGIN || "*", methods: ["GET",
"POST"] },
  path: "/socket.io/",
});
```

Configuració del servidor WebSocket i CORS.

Fitxer: index.js

Seccions Clau

1. Connexió i “rooms”

```
socket.on("join", (data) => {
  const room = `user.${data.userId}`;
  socket.join(room);
});
```

Cada usuari entra al seu room personal per notificaciones.

2. Missatges P2P (flux principal)

```
socket.on("send-message", async (data, callback) => {
  const response = await fetch(`${apiUrl}/chat/messages`, { method:
"POST", ... });
  const result = await response.json();
  io.to(chatRoom).emit("new.message", { ...messageData, tempId });
});
```

Envia missatge a l'API i després emet via sockets.

3. Redis Broadcast (Laravel → [Socket.io](https://socket.io/))

```
redisSubscriber.psubscribe(pattern, () => {});
redisSubscriber.on("pmessage", (_p, channel, rawMessage) => {
  const payload = JSON.parse(rawMessage);
  io.to(laravelChannel).emit(payload.event, payload.data);
});
```

Traducció dels events de Laravel a Socket.io.

Fitxer: [index.js](#)

Gestió de Dades

- **Fonts:** events del client + API Laravel + Redis broadcast.
- **Persistència:** la fa Laravel (el socket només envia/rep).
- **Sincronització:** rooms per usuari, post, chat i grup.

Mètodes Principals

- join, join-post, join-chat, join-group
- send-message, mark-read
- call-user, make-answer, ice-candidate (trucades)

Característiques Visuals

No aplica (backend). Equivalent: logs de connexió i errors.

Funcionalitats Destacades

- Temps real per missatges i notifikacions.
- Integració amb Redis per events broadcast del backend.
- Rooms segmentades per usuari, post, chat i grup.

AI-moderation (microservei):

Plantilla Principal

Arrencada del servidor Express

```
const app = express();
app.use(express.json({ limit: '512kb' }));

const PORT = Number(process.env.PORT || 8088);

app.listen(PORT, () => {
  console.log(`ai-moderation listening on port ${PORT}`);
});
```

Servei Node amb Express que exposa endpoints de salut i moderacio.
Fitxer: index.js

Seccions Clau

1. Configuracio de models i llindars

```
const MODEL_ID = process.env.AI_MODEL_ID || 'Xenova/toxic-bert';
const BLOCK_THRESHOLD = Number(process.env.AI_BLOCK_THRESHOLD || 0.72);
const MEDIUM_THRESHOLD = Number(process.env.AI_MEDIUM_THRESHOLD || 0.45);
```

Defineix models i llindars de decisió.

2. Endpoint de salut

```
app.get('/health', (_req, res) => {
  res.json({ status: 'ok', service: 'ai-moderation' });
});
```

Permet verificar estat del servei i models carregats.

3. Endpoint de moderacio

```
app.post('/moderate', checkApiKey, async (req, res) => {
  const text = normalizeText(req.body?.content, req.body?.code_snippet);
  // ... aplica LLM, model i zero-shot
  return res.json(decision);
});
```

Endpoint principal que retorna la decisió de moderació.

Gestió de Dades

1. Normalització de text

```
function normalizeText(content, codeSnippet) {
  const joined = `${contentPart}\n${codePart}`.trim();
  return joined.length <= MAX_TEXT_LENGTH ? joined : joined.slice(0,
MAX_TEXT_LENGTH);
}
```

Unifica contingut i codi, limita longitud.

2. Sortida normalitzada

```
return {
  allowed: true,
  score: 0.4,
  severity: 'medium',
  reason: 'Content accepted by AI model.',
  categories: [],
};
```

La resposta sempre segueix el mateix format.

Metodes Principals

1. Carrega de model principal

```
async function getClassifier() {  
  
  classifierPromise = pipeline('text-classification', MODEL_ID, {  
    quantized: true });  
  
  return classifierPromise;  
}
```

2. Zero-shot semantic classifier

```
async function analyzeWithZeroShot(text) {  
  const result = await zeroShot(text, HARMFUL_LABELS, { multi_label:  
    true });  
  // calcula score i allowed  
}
```

3. Moderacio amb LLM (opcional)

```
async function moderateWithLlm(text) {  
  const response = await fetch(`${LLM_BASE_URL}/chat/completions`, { ...  
});  
  // parse JSON i retorna decisio  
}
```

Caracteristiques Visuals

No aplica (microservei). Equivalent: logs i respostes JSON.

Funcionalitats Destacades

- **Decisio combinada:** model + zero-shot amb regla de max score.
- **Fallbacks:** si un motor falla, intenta un altre.
- **Seguretat:** API key opcional amb X-API-Key.
- **LLM opcional:** si AI_LLM_ENABLED=true, pot delegar a model extern.

Dockers:

Visió general del stack

El desplegament utilitza Docker Compose i divideix el sistema en serveis:

- webserver (Nginx): reverse proxy y entrada unica
- api (Laravel/PHP-FPM)
- client (React/Vite)
- socket (Socket.io)
- ai-moderation (Node + ML)
- mysql
- redis
- mailpit (solo dev)
- adminer (solo dev)
- queue (solo prod)
- certbot (solo prod)
- turn (coturn) (solo prod)

Definició completa: docker-compose.dev.yml i docker-compose.prod.yml

Desplegament a DESENVOLUPAMENT

1. Inicialització automàtica

Script recomanat:

```
chmod +x init-dev.sh
./init-dev.sh
```

L'script:

- Crea .env, .env i .env
- Construeix imatges
- Aixeca contenidors
- Instal·la dependències
- Executa migracions i seeders
- Mostra URLs i credencials
-

Veure: init-dev.sh

2. Serveis i ports (dev)

- Web (Nginx): http://localhost:8080
- API: http://localhost:8080/api
- Vite: http://localhost:5173

- Socket.io: ws://localhost:8080/socket.io
- Adminer: http://localhost:8081
- Mailpit: http://localhost:8025
- MySQL: localhost:3306
- Redis: localhost:6379

3. Routing Nginx (dev)

- /api/* → PHP-FPM (Laravel)
- /socket.io/* → Socket.io
- /storage/* → archivos estáticos del backend
- / → Vite dev server

Veure: default.dev.conf

Desplegament a DESENVOLUPAMENT

1. Build i arrencada

```
docker compose -f docker-compose.prod.yml up --build -d
```

En prod:

- El frontend es builda i copia a volum client_build
- Nginx serveix estàtics des de client_build
- L'API s'executa en mode production
- Redis i MySQL en mode persistent
- S'activa queue per a treballs en background

Veure: docker-compose.prod.yml

2. SSL amb Let's Encrypt

Script recomanat:

```
./init-ssl.sh tu-dominio.com tu-email@ejemplo.com
```

- Nginx temporal només HTTP
- Demana certificat amb Certbot
- Configura DOMAIN a .env
- Aixeca stack complet amb HTTPS

Veure: init-ssl.sh

3. Routing Nginx (prod)

- HTTP 80 → redirigeix a HTTPS
- /api/* → Laravel (PHP-FPM)
- /socket.io/* → Socket.io (WSS)
- /storage/* → fitxers pujats
- / → React SPA (fallback a index.html)

Veure: default.prod.conf

Persistència i dades

Volums principals

- **mysql_data** → dades MySQL
- **redis_data** → dades Redis
- **api_storage** → storage Laravel (uploads)
- **client_build** → build de React
- **certbot_www** → ACME challenges SSL

Inicialització MySQL

```
CREATE DATABASE IF NOT EXISTS `tfg_database`;  
CREATE USER IF NOT EXISTS 'tfg_user'@'%' IDENTIFIED BY 'tfg_password';
```

Veure: init.sql

Ordres útils

Logs

```
docker compose -f docker-compose.dev.yml logs -f
```

Parar stack

```
docker compose -f docker-compose.dev.yml down
```

Artisan

```
docker compose -f docker-compose.dev.yml exec api php artisan
```